CYLite Compiler Project

Language Specification and Grammar Modifications Report

Authors:

Tejas Lohia
[Shardul Junagade / 23110297]
[Swayam Borate / 23110066]

CS327 Compilers - Lab Assignment

March 30, 2026

Contents

1 Introduction **2**

2 Grammar Modifications **3**
2.1 Lexical Additions (Tokens) 3
2.2 The `elif` Conditional Construct 3
2.3 Native Error Handling: `try-except` 4
2.4 Ergonomic Iteration: `foreach` and `in range` 4
2.5 The explicit `pass` Operation 4
2.6 First-Class `print` Built-in 5
2.7 Results 5

3 Automaton Generation and Initial Analysis **6**
3.1 Bison Invocation and Outputs 6
3.2 Initial LALR(1) Automaton Analysis 6
3.3 Conflict Identification and Resolution 6
3.4 Shift/Reduce Conflicts: The Dangling-Else and `elif` Ambiguity 7
3.5 Reduce/Reduce Conflict Analysis 7
3.6 Conflict Resolution and Grammar Modifications 7

1 Introduction

The landscape of programming languages has long been divided between highly performant, statically typed systems languages like C, and highly expressive, dynamically typed scripting languages like Python. C provides unparalleled control over hardware and memory layout, executing with optimal efficiency. However, its syntax can be unforgiving; error handling relies heavily on cumbersome return codes, control flow can become deeply nested and difficult to read, and iterative constructs often lead to off-by-one errors. Conversely, Python offers ergonomic syntax, explicit error handling blocks, and intuitive looping mechanisms, but at the cost of execution speed and strict compile-time type safety.

CYLite represents a modern hybridization of these two paradigms. It is designed as an extension of the YAPL (Yet Another Programming Language) baseline compiler, effectively injecting Python-inspired syntactic sugar and control flow structures into a C-style language grammar. The fundamental motivation behind CYLite is to provide developers with the readability and expressiveness of modern high-level languages without sacrificing the structural rigor of a statically compiled environment.

Developing a formal grammar for CYLite requires adapting these high-level programming concepts into a strictly unambiguous LALR(1) framework compatible with compiler generation tools like GNU Bison. By integrating these features directly at the lexical and grammatical level—rather than relying on standard library macros or external subroutines—the compiler pipeline can elevate them to first-class citizens in the language. This allows the compiler front-end to perform rigorous syntax checking, generate more optimized Abstract Syntax Trees (ASTs), and produce clearer compile-time error messages tailored to the programmer's explicit intent.

This project focused heavily on redefining the parser rules (`yapl.y`) to accommodate these new grammatical structures. By modifying the base YAPL grammar, we introduced several new nonterminal rules and shifted existing expressions to handle newly introduced keywords. Specifically, the grammar now natively understands chained conditional blocks (via `elif`), explicit null operations (via `pass`), native I/O primitives (`print`), ergonomic loop iterators (`foreach` and `in range`), and block-scoped error encapsulation (`try-except`).

This structural evolution ensures that CYLite can parse declarative flow states seamlessly while resolving dangling-else ambiguities and iteration pitfalls inherent to older grammars. This report breaks down the philosophy behind each addition and details the

exact grammatical transformations applied to achieve the CYLite language specification.

2 Grammar Modifications

To implement the CYLite language extensions, the lexical analyzer (generated via Flex) was first updated to recognize new keywords and allocate corresponding tokens. The parser (`yapl.y`) was subsequently heavily modified to integrate these new tokens into valid syntax trees.

2.1 Lexical Additions (Tokens)

We introduced the following terminal tokens into the parser to represent the core vocabulary of the new language features:

- `ELIF`, `PASS`, `TRY`, `EXCEPT`, `PRINT`, `RANGE`, `IN`, `FOREACH`

```
int ifs_wo_else=0;
int ladder_len=0,hold=0;
int max=-1;
%}

%token  ELIF PASS TRY EXCEPT PRINT RANGE IN FOREACH
%token  IDENTIFIER I_CONSTANT F_CONSTANT STRING_LITERAL FUNC_NAME SIZEOF
%token  PTR_OP INC_OP DEC_OP LEFT_OP RIGHT_OP LE_OP GE_OP EQ_OP NE_OP TH_OP
%token  AND_OP OR_OP MUL_ASSIGN DIV_ASSIGN MOD_ASSIGN ADD_ASSIGN
%token  SUB_ASSIGN LEFT_ASSIGN RIGHT_ASSIGN AND_ASSIGN
```

Figure 1: Implementation of the new CYLite tokens in the lexical analyzer.

2.2 The `elif` Conditional Construct

In standard C, chaining conditionals requires either deeply nested `else { if (...) }` blocks or the '`else if`' syntax, which is grammatically just an `else` block containing a single `if` statement. In complex logical branching, this can disrupt readability and complicate AST generation due to classic dangling-else ambiguities. Python's `elif` provides a unified, first-class construct for multi-way branching without artificial nesting.

Grammar Change: We defined a new nonterminal `elif_list` to explicitly handle zero or more `elif` branches recursively. We then expanded `selection_statement` to support an `if` block followed optionally by an `elif_list` and a final optional `else`.

```
elif_list
    : ELIF '(' expression ')' statement
    | elif_list ELIF '(' expression ')' statement
    ;

selection_statement
    : IF '(' expression ')' statement ELSE statement
    | IF '(' expression ')' statement
    | IF '(' expression ')' statement elif_list ELSE statement
    | IF '(' expression ')' statement elif_list
    ...
```

2.3 Native Error Handling: `try-except`

Error handling in C typically falls back to checking integer return codes after every function call, or using precarious `setjmp/longjmp` chains for global exits. Modern languages isolate fragile code within explicit boundaries, redirecting failures to localized catch blocks. By implementing `try-except` directly in the CYLite grammar, we establish dedicated scope blocks for execution and recovery, allowing the backend to map these to structured, object-level exception-handling routines.

Grammar Change: We introduced a `try_except_statement` combining the `TRY` keyword, a localized block (`compound_statement`), the `EXCEPT` keyword, and its associative block. This nonterminal was then injected into the overarching `statement` rule.

```
try_except_statement
    : TRY compound_statement EXCEPT compound_statement
    ;

statement
    ...
    | try_except_statement
```

2.4 Ergonomic Iteration: `foreach` and `in range`

Classic C `for` loops require boilerplate initialization, condition checking, and manual incrementation (e.g., `for(int i=0; i<10; i++)`). This pattern is verbose and inherently prone to off-by-one errors. CYLite adopts `foreach id in enumerable` and `for(id in range(a, b))` to abstract the iteration boilerplate. This makes the programmer's intent purely declarative rather than imperative, preventing out-of-bounds complications.

Grammar Change: The `iteration_statement` nonterminal was expanded to natively parse these specialized loops, tying identifiers directly to iterable sequences and mathematical ranges.

```
iteration_statement
    ...
    | FOR '(' IDENTIFIER IN RANGE '(' expression ',' expression ')' ')' statement
    | FOREACH '(' IDENTIFIER IN expression ')' compound_statement
```

2.5 The explicit `pass` Operation

In standard C, an intentionally empty block or stubbed function is written as an empty curly block {} or a lone semicolon `;`. This is visually ambiguous—compilers and semantic linters regularly flag it as a probable mistake. Promoting the explicit `pass` keyword provides an unambiguous syntactic placeholder, asserting that the empty block is intentional.

Grammar Change: `PASS ';'` was added directly as a terminal production alternative within the `statement` rule.

```
statement
    ...
    | PASS ';'
```

2.6 First-Class `print` Built-in

Rather than relying exclusively on importing `stdio.h` to use the variadic `printf` function—which requires cumbersome format strings and type awareness—CYLite promotes basic output to a native built-in language feature. By integrating it into the grammar, `print` can be more strictly type-checked or implicitly optimized for polymorphic output at compile-time by the parser's AST builder without requiring external linkage headers.

Grammar Change: The `print` construct was appended defensively to `postfix_expression`, supporting both empty calls and list arguments.

```
postfix_expression
    ...
    | PRINT '(' ')'
    | PRINT '(' argument_expression_list ')'
```

2.7 Results

The integration of these advanced features required careful consideration of LALR(1) parsing constraints to avoid shift/reduce and reduce/reduce conflicts within the Bison generator framework. By directly extending the `yapl.y` grammar definitions, CYLite successfully bridges the architectural gap between systems programming and scripting conveniences. Developers can now write highly readable, python-inspired code while leaning on the reliable, rigorous syntax-checking rules inherent to a C-like toolchain.

3 Automaton Generation and Initial Analysis

To construct the LALR(1) item sets and build the parser automaton (fulfilling Part 1 of the assignment requirements), we processed the modified `yapl.y` grammar file using GNU Bison.

3.1 Bison Invocation and Outputs

The parser generator was invoked using the following command:

```
$ bison -v -d yapl.y
```

This command utilizes two critical flags:

- **`-d`:** Generates the `yapl.tab.h` header file, which exposes the newly defined CYLite terminal tokens (e.g., `ELIF`, `FOREACH`, `TRY`) to the Flex lexical analyzer.
- **`-v` (Verbose):** Instructs Bison to generate a detailed `yapl.output` file. This file is crucial as it contains the human-readable representation of every LALR(1) item set and the complete state transition graph for the automaton.

3.2 Initial LALR(1) Automaton Analysis

The generation process successfully constructed the LALR(1) automaton required to parse the CYLite language specification. An initial analysis of the generated `yapl.output` report reveals the following structural metrics regarding the grammar's complexity:

- **Grammar Rules:** The parser implements over 130 distinct production rules to encapsulate both the core C-style expressions and the advanced CYLite extensions.
- **Item Sets (States):** The resulting LALR(1) automaton consists of 522 distinct states (item sets). This large state space is a direct result of the complex expression precedence and the newly introduced deeply nested control structures (such as `try-except` and `elif` ladders).

While the item sets and state machine were successfully compiled, the integration of optional branching (specifically the `elif` and `else` derivations) and ambiguous operator precedence introduced non-determinism into the parsing table. The initial Bison compilation reported exactly **6 shift/reduce conflicts** and **1 reduce/reduce conflict**. The identification, isolation, and resolution of these conflicts are documented in the following section.

3.3 Conflict Identification and Resolution

As noted in the initial automaton analysis, the introduction of CYLite's advanced control structures—specifically the `elif` ladders and optional `else` blocks—introduced non-determinism into the LALR(1) parsing table. When processed by GNU Bison, the grammar generated exactly 6 shift/reduce conflicts and 1 reduce/reduce conflict.

To satisfy the structural rigor required by the compiler front-end, we must isolate the specific item sets where the automaton encounters multiple valid transition paths, identify the conflicting productions, and apply deterministic precedence rules to resolve them.

3.4 Shift/Reduce Conflicts: The Dangling-Else and `elif` Ambiguity

3.5 Reduce/Reduce Conflict Analysis

3.6 Conflict Resolution and Grammar Modifications